

```
1
2 // COS30008, Problem Set 4, Problem 3, 2022
3
4 #pragma once
5
6 #include "BinarySearchTree.h"
7
8 #include <exception>
9 #include <stack>
10 #include <stdexcept>
11
12 template <typename T> class BinarySearchTreeIterator {
13     private:
14         using BSTree = BinarySearchTree<T>;
15         using BNode = BinaryTreeNode<T>;
16         using BTreeNode = BNode *;
17         using BTNStack = std::stack<BTreeNode>;
18
19         const BSTree &fBSTree; // binary search tree
20         BTNStack fStack;       // DFS traversal stack
21
22         void pushLeft(BNode *aNode) {
23             while (!aNode->empty()) {
24                 this->fStack.push(aNode);
25                 aNode = aNode->left;
26             }
27         };
28
29     public:
30         using Iterator = BinarySearchTreeIterator<T>;
31
32         BinarySearchTreeIterator(const BinarySearchTree<T> &aBSTree)
33             : fBSTree(aBSTree) {}
34
35         const T &operator*() const {
36             if (this->fStack.empty()) {
37                 throw std::out_of_range("Empty root");
38             }
39             return this->fStack.top()->key;
40         }
41
42         Iterator &operator++() {
43             // if node has a left node, go into that node
44
45             if (!this->fStack.empty()) {
46                 // travel root?
47                 BTreeNode current_root = this->fStack.top();
48                 this->fStack.pop();
49
```

```
50         if (!current_root->right->empty()) {
51             this->pushLeft(current_root->right);
52         }
53     }
54
55     return *this;
56 };
57
58 Iterator operator++(int) {
59     Iterator old = *this;
60     ++(*this);
61     return old;
62 }
63 bool operator==(const Iterator &aOtherIter) const {
64     // check if two stacks are equal maybe?
65     return ((this->fStack.empty() && aOtherIter.fStack.empty()) ||
66            (this->fStack.size() == aOtherIter.fStack.size() &&
67             this->fStack.top()->key == aOtherIter.fStack.top()->key));
68 };
69 bool operator!=(const Iterator &aOtherIter) const {
70     return !(*this == aOtherIter);
71 };
72
73 Iterator begin() const {
74     // YOU CAN JUST DO THIS?
75     Iterator it = Iterator(this->fBSTree);
76     if (!this->fBSTree.empty()) {
77         it.pushLeft(this->fBSTree.fRoot);
78     }
79     return it;
80 };
81 Iterator end() const { return Iterator(this->fBSTree); };
82 };
83
```